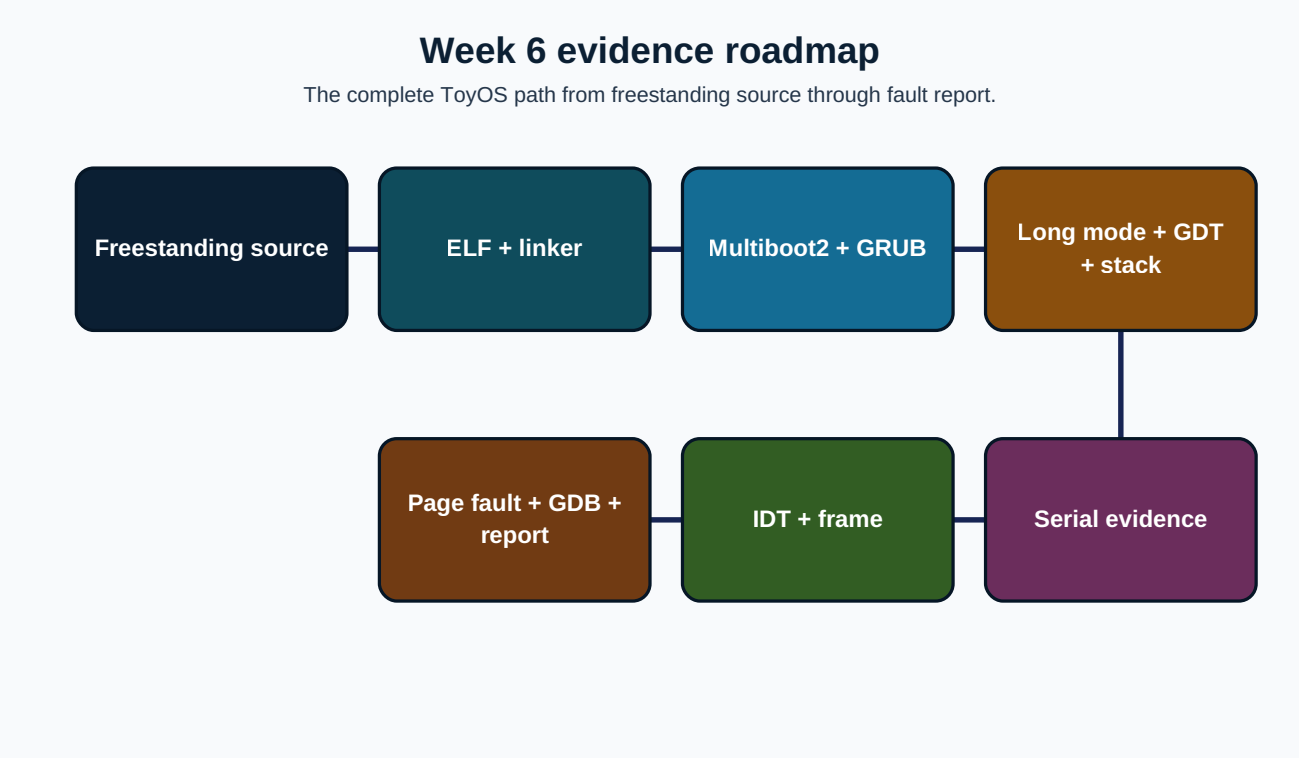
Week 6 — ToyOS Bootstrap and Exceptions

Format: seven daily sections collected into one mastery package

Suggested workload: approximately 21 focused hours, adjustable to mastery

> A kernel starts with fewer guarantees than an ordinary C program. Make each guarantee explicit, then preserve enough evidence to debug when one fails.



Roadmap from freestanding source to an evidence-bounded fault report.

This week follows the existing freestanding x86-64 kernel in `projects/toyos`. You will trace GRUB's Multiboot2 handoff, the 32-bit bootstrap, long-mode entry, serial diagnostics, exception entry, TSS/IST stack selection, and symbol-aware fault diagnosis. The package does not duplicate the kernel: each practical challenge invokes or inspects that canonical project.

The recurring discipline is:

```
purpose → prediction → run or inspect → preserve evidence → explain limits
```

“Week” names a coherent mastery module, not a deadline. Do not treat a boot banner as proof of every preceding claim. Tie each conclusion to source, ELF metadata, serial output, QEMU status, or debugger state.

Responsibility map

- **C language:** object representations, pointer and integer operations, `volatile`, undefined behavior, and the freestanding implementation.
- **ABI/toolchain:** argument registers, stack alignment, object format, relocations, symbol names, and compiler options.
- **x86-64 architecture:** control registers, page-table interpretation, descriptor tables, exception vectors, error codes, and `iretq`.
- **ToyOS implementation:** chosen mappings, stacks, logging, frame layout, fault policy, tests, and QEMU exit protocol.

An architecture specifies mechanisms; an ABI lets separately built pieces cooperate; this kernel chooses one implementation and policy.

Daily roadmap

Day 1 — Freestanding C and the project contract

Read the build flags and kernel entry contract. Explain what `-ffreestanding`, `-nostdlib`, `-mno-red-zone`, and the project header do—and do not—guarantee.

- Lesson: day-001-freestanding-c-and-project-contract.md
- Challenge: `challenges/day-001-project-contract.sh`
- Evidence: compiler/linker flags, undefined-symbol inspection, and a clean Week 6 build without changing canonical source.

Day 2 — ELF, linker, Multiboot2, and GRUB

Follow the image from linker script to Multiboot2 validation and ISO layout.

- Lesson: day-002-elf-linker-multiboot2-and-grub.md
- Challenge: `challenges/day-002-elf-boot-contract.sh`
- Evidence: ELF header/sections/program headers/symbols, Multiboot2 recognition, and the GRUB configuration actually packaged in the ISO.

Day 3 — From 32-bit entry to 64-bit C

Trace initial page tables, PAE, EFER.LME, paging, GDT selection, far jump, stack setup, and the System V AMD64 argument handoff.

- Lesson: day-003-long-mode-pages-gdt-and-stack.md
- Challenge: `challenges/day-003-long-mode-trace.sh`
- Evidence: annotated bootstrap disassembly and the normal serial regression.

Day 4 — Serial, panic, and assertions

Treat COM1 as the early diagnostic channel and separate an assertion policy from the architecture's exception machinery.

- Lesson: day-004-serial-panic-and-assert.md
- Challenge: `challenges/day-004-serial-panic-assert.sh`
- Evidence: normal and expected-assert serial markers plus host exit status.

Day 5 — IDT, stubs, and one saved frame

Connect an IDT gate to assembly stubs, normalize hardware error-code differences, and verify the C structure against push order.

- Lesson: day-005-idt-stubs-and-uniform-frame.md
- Challenge: `challenges/day-005-exception-frame.sh`
- Evidence: breakpoint self-test, divide-error test, and frame-layout inspection.

Day 6 — TSS, IST, page faults, and GDB

Explain why a known-good exception stack matters, decode `CR2` and the page fault error code, and correlate saved `RIP` with symbols and source.

- Lesson: day-006-tss-ist-page-fault-and-gdb.md
- Challenge: `challenges/day-006-page-fault-diagnosis.sh`
- Evidence: expected page-fault regression, static symbolization, and a generated command guide for an optional live GDB session.

Day 7 — Integrated fault-debug report

Produce one evidence-bounded report and compare ToyOS's exception path with xv6 concepts and current Linux source landmarks.

- Lesson: day-007-integrated-fault-debug-report.md
- Challenge: `challenges/day-007-fault-report.sh`
- Evidence: a reproducible report containing build identity, relevant source excerpts, ELF symbols, all Week 6 test markers, and explicit limits.

Integrated mastery checkpoint

Starting at the deliberate unmapped write in `projects/toyos/src/kernel.c`, explain:

```
C access
  → compiler-selected faulting instruction
  → page translation failure
  → vector 14 gate and IST selection
  → assembly register/error-code normalization
  → struct interrupt_frame
  → CR2 and error-bit decoding
  → serial evidence and expected QEMU exit
  → symbol/source correlation in GDB
```

For each arrow, identify architecture, ABI, or ToyOS implementation. Then state what the evidence does not prove.

References and version discipline

The repository's architecture comparison is a local orientation aid. Optional local source trees under `references/library/` are examples, not normative specifications. For exact exception, paging, and descriptor semantics, consult the current Intel 64 and IA-32 Software Developer's

Manual or AMD64 Architecture Programmer's Manual. For comparison, use current Arm A-profile and RISC-V privileged architecture manuals. For Linux claims, record the source revision you inspected; paths and details change.

Do not auto-fill confidence, completion, hours, or mastery. Those remain the learner's evidence-based assessment.

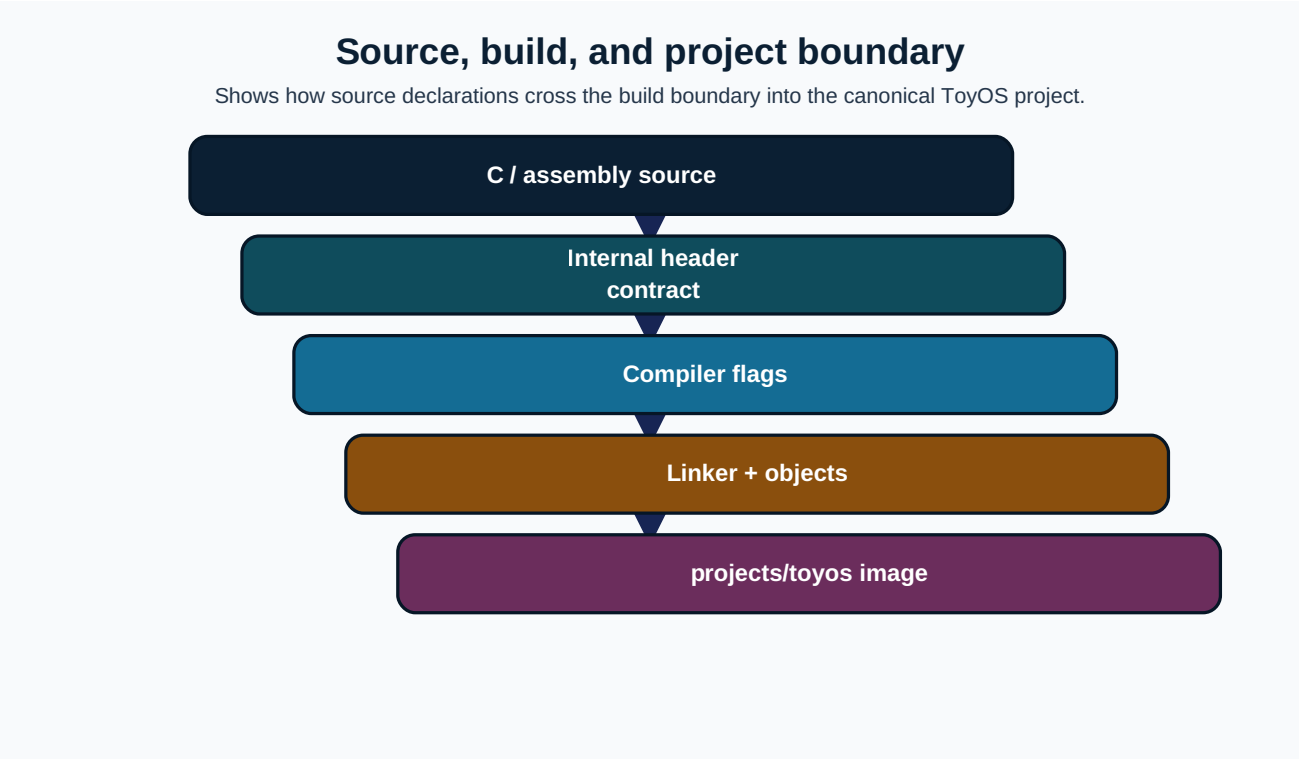
Day 1 — Freestanding C and the Project Contract

Target time: approximately 2½–3 hours

> Before `kernel_main` can print one byte, which guarantees come from C, which come from the compiler ABI, and which has ToyOS built for itself?



Contrasts inherited hosted services with the explicit ToyOS freestanding contract.



Shows how source declarations cross the build boundary into the canonical ToyOS project.

An ordinary Linux process inherits a loader, virtual address space, stack, C runtime, file descriptors, and system-call boundary. ToyOS begins under a bootloader with none of libc's hosted environment. “Freestanding” is not a synonym for “anything goes”: the C abstract machine still matters, while the implementation must provide the environment around it.

---

1. Read the contract before the code

The canonical project is `projects/toyos`. Its Week 6 build compiles four C and three assembly files, but milestone guards keep the memory and scheduler features inactive. The important boundaries are:

```
GRUB/Multiboot2 handoff
  → assembly bootstrap owns initial machine state
  → kernel_main uses a compiler calling convention
  → arch_init installs ToyOS exception machinery
  → serial output and QEMU exit provide test evidence
```

`include/toyos.h` is an internal interface contract. It declares fixed-width types, the saved exception frame, assembly symbols, and functions shared across translation units. It does not define what the CPU pushes on exception entry; the architecture does. It expresses how this implementation exposes that state to C.

2. Hosted versus freestanding C

C distinguishes hosted and freestanding implementations. In a hosted environment, program startup reaches `main` with a substantial standard library available. In this freestanding build:

- the entry symbol is `_start`, not `main`;

- no C runtime initializes the process;
- `-nostdlib` prevents the linker from supplying normal startup and libraries;
- kernel code provides its own output, panic, and termination mechanisms;
- some library-shaped operations could still be emitted by a compiler unless options and code generation are controlled.

`-ffreestanding` tells GCC that a hosted environment cannot be assumed. `-fno-builtin` additionally prevents ordinary builtin substitution. Neither option initializes page tables, chooses an ABI, or makes undefined C behavior safe.

The kernel uses `stdint.h`, `stddef.h`, and `stdbool.h`. Freestanding does not mean “no headers”; a compiler supplies the freestanding headers required by its implementation.

3. Why these machine options exist

Read `projects/toyos/Makefile` as an executable specification.

- `-m64` selects x86-64 code generation after the bootstrap reaches long mode.
- `-mno-red-zone` prevents compiler use of the 128 bytes below `RSP`. An asynchronous exception can otherwise overwrite compiler-owned temporaries.
- `-mgeneral-regs-only` avoids generated floating-point/vector register use before the kernel has established save/restore policy for that state.
- `-mcmodel=kernel` chooses an x86-64 addressing model suitable for a kernel image; it is a code-generation choice, not a mapping policy.
- `-fno-pie` avoids position-independent executable assumptions.
- `-fno-stack-protector` avoids references to a protector runtime the project has not implemented.
- `-fno-asynchronous-unwind-tables` omits unwind tables that this milestone does not consume.

The warning set ends in `-Werror`. That improves build discipline but does not prove absence of memory errors, ABI mistakes, or architectural faults.

4. Language, ABI, architecture, implementation

Use this classification whenever a claim feels ambiguous:

Claim	Owner
Dereferencing an invalid C pointer has undefined C behavior	C language
First two integer arguments arrive in `RDI` and `RSI`	System V AMD64 ABI
`CR2` records the page-fault linear address	x86-64 architecture
ToyOS prints `PAGE-FAULT address=` over COM1	ToyOS implementation

The deliberate page-fault injection converts a controlled implementation test into architecture evidence. It must not be generalized into a claim that arbitrary undefined C behavior reliably raises a page fault.

The bootstrap passes two 32-bit values in `EDI` and `ESI` before calling:

```
void kernel_main(uint32_t magic, uint32_t multiboot_info);
```

That register choice is ABI. The meanings of the values come from the Multiboot2 contract. Their checks and panic response are ToyOS policy.

5. A narrow runtime surface

`arch.c` implements serial output without allocation, formatting libraries, or system calls. `serial_hex` and `serial_dec` are deliberately bounded. `panic_at` prints a message and source location, then exits QEMU. The macro:

```
#define ASSERT(condition) \
    do { if (!(condition)) { PANIC("assertion failed: " #condition); } } while (0)
```

is implementation policy. It evaluates `condition` once and reports the expansion-site file and line. It is not the hosted `<assert.h>` facility, and it does not recover.

6. Practical challenge

Use `challenges/day-001-project-contract.sh` from the repository root.

Purpose: inspect the exact compile/link contract and verify that the existing Week 6 profile links without hosted startup libraries.

Predict: write down which entry symbol you expect and whether the final ELF should have unresolved symbols. Predict one flag whose removal could silently compile but violate an interrupt-time assumption.

Run: the wrapper prints selected Makefile lines, builds the canonical profile, then asks `readelf` about the entry and symbol table. It does not modify `projects/toyos`.

Evidence: preserve the entry address, `_start` and `kernel_main` symbols, and the undefined-symbol count. A zero count proves only that this ELF's static symbol references were resolved—not that every runtime path is valid.

7. Architecture lens at a real boundary

Freestanding C is portable; bootstrap and exception ABIs are not.

- x86-64 reaches C here through a custom 32-to-64-bit assembly path and uses the System V AMD64 register convention.
- AArch64 firmware or a boot protocol commonly enters at an exception level; an assembly entry establishes `SP`, clears required storage, and passes arguments in `x0`–`x7` under AAPCS64.
- RISC-V firmware commonly hands off in S-mode or M-mode with arguments in `a0`/`a1`; the psABI governs C calls, while privileged state governs traps.

These are boundary comparisons, not claims that all platforms boot alike.

Mastery: Explain · Observe · Build

Explain

1. Why does `-ffreestanding` not remove the C abstract machine?
2. Why is `-mno-red-zone` connected to asynchronous entry?
3. Why are Multiboot2 argument meaning and AMD64 register placement separate contracts?
4. Why is a successful link weaker evidence than a successful fault test?

Observe

Annotate one line each from the Makefile, header, bootstrap, and kernel entry. Label every line C, ABI, architecture, or ToyOS implementation.

Build

Without editing canonical code, write a one-page “entry contract” containing: preconditions at `_start`, postconditions before `kernel_main`, callable services available in C, and unsupported assumptions. Every statement should cite a source path or observed artifact.

Evidence limits

Static inspection cannot prove that GRUB loads the image, that page tables are accepted, or that serial I/O works. Those claims need later dynamic evidence.

Now check your understanding

You have finished the theory for this section. Before moving on, use the quick check below to find anything that still feels uncertain, then reinforce the idea with the practical lab when one is provided.

Quick theory check

Answer from memory first. Explanations appear only after you submit your choices.

- Start the interactive theory check
- View quiz source on GitHub

Put it into practice — Project Contract

Format: shell lab

What you will do: Inspect and validate the canonical ToyOS freestanding build contract.

Predict the important result first, then run the lab and explain any difference between prediction and observation.

- Open the lab file

Copy and run:

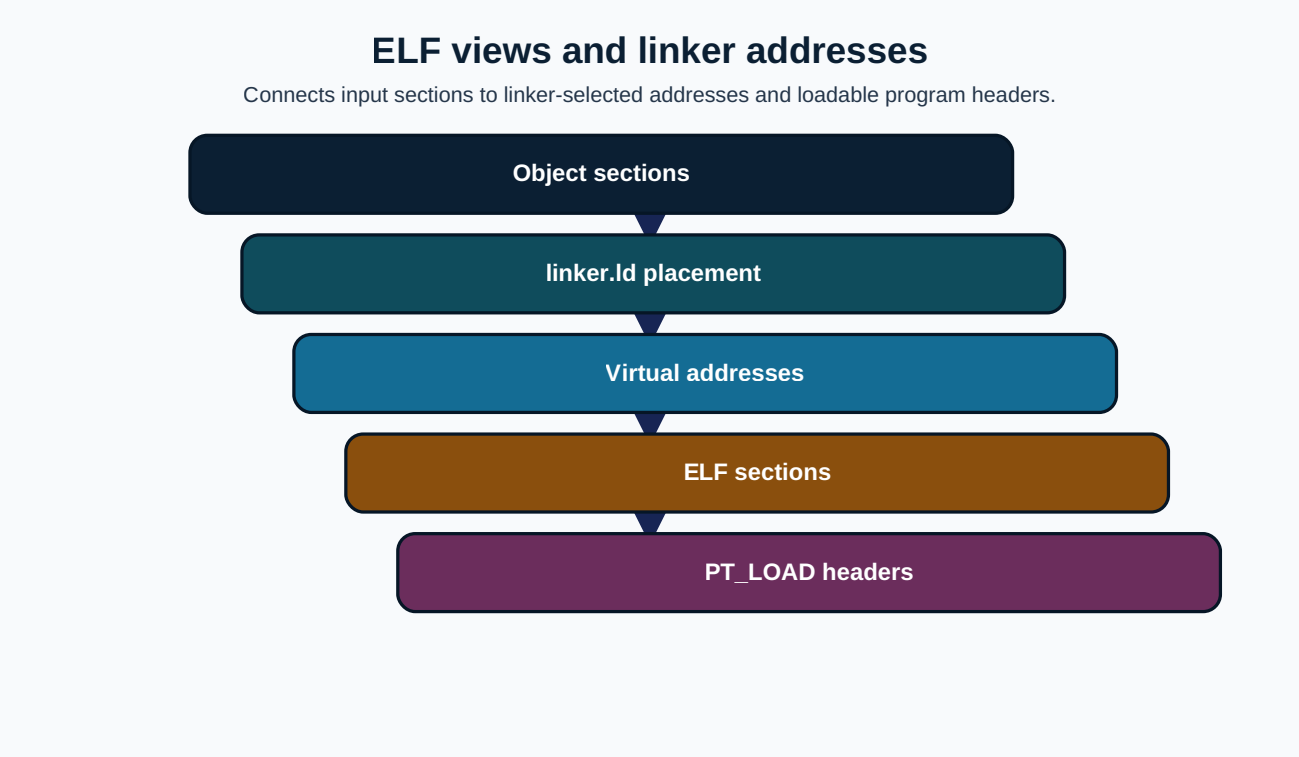
```
bash 06-toyos-bootstrap-and-exceptions/challenges/day-001-project-contract.sh
```

- View lab source on GitHub

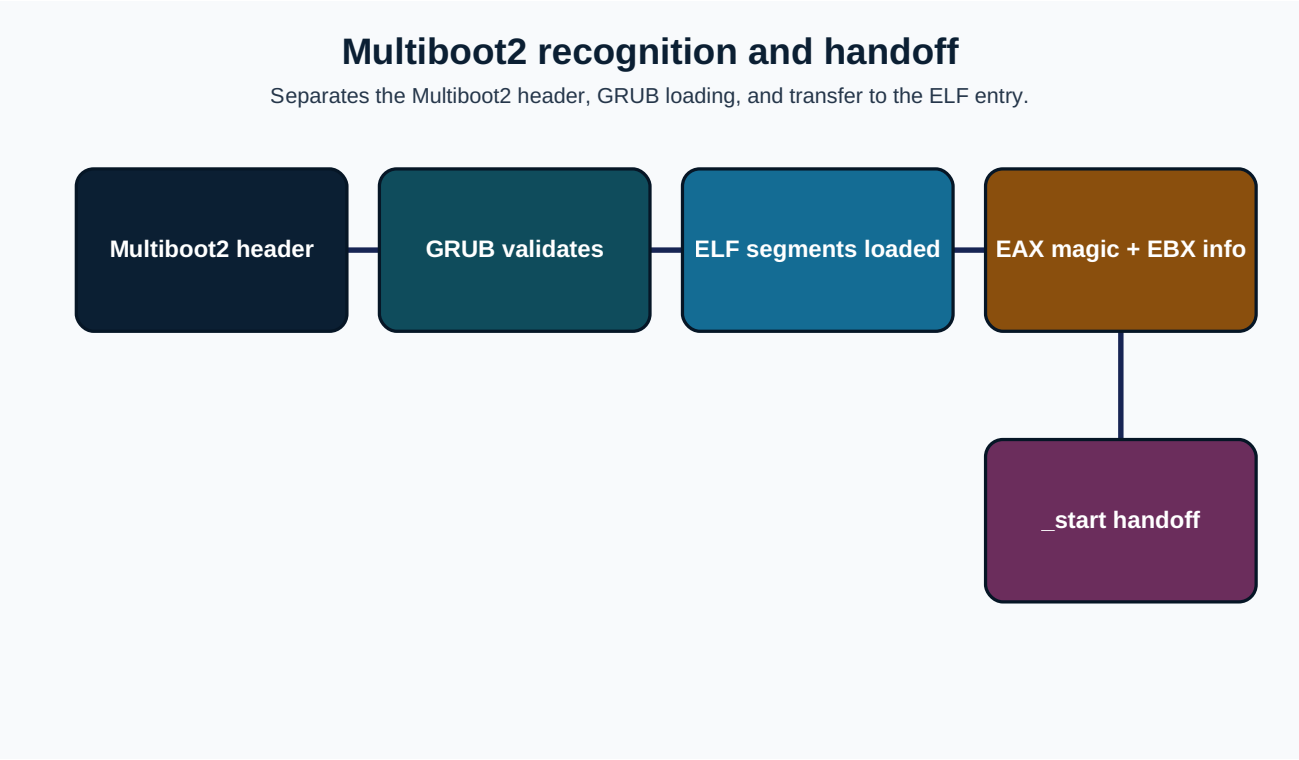
Day 2 — ELF, Linker, Multiboot2, and GRUB

Target time: approximately 2½–3 hours

> The compiler produced object files. What makes the result a kernel image that GRUB can recognize rather than merely another ELF file?



Connects input sections to linker-selected addresses and loadable program headers.



Separates the Multiboot2 header, GRUB loading, and transfer to the ELF entry.

The ELF container, linker script, Multiboot2 header, GRUB configuration, and ISO each answer a different question. Keep them separate.

---

1. The artifact pipeline

```
C/assembly source
  → relocatable ELF objects
  → linker.ld arranges one executable ELF
  → Multiboot2 header advertises the boot contract
  → grub2-file validates recognition
  → grub2-mkrescue packages ELF + grub.cfg into an ISO
  → GRUB loads and transfers control to ELF entry _start
```

ELF describes sections, segments, symbols, addresses, and an entry point. ELF alone does not require a Multiboot2-aware loader. The header in `src/boot.S` is what lets GRUB recognize this kernel under that protocol.

2. What the linker script chooses

`projects/toyos/linker.ld` says:

```
ENTRY(_start)
. = 1M
.multiboot2 → .text → .rodata → .data → .bss
```

The image begins at link address 1 MiB. The Multiboot2 input section is kept and aligned to eight bytes. Code and later sections receive page-oriented alignment. `.bss` is `NOLOAD`: the ELF describes its memory extent without storing all zero bytes in the file.

This is implementation policy expressed to the linker. The CPU does not know ELF section names. GRUB and the linker interpret the file; after loading, the processor fetches bytes at addresses.

The symbols `__kernel_start` and `__kernel_end` mark an interval that later memory code can reserve. Their values come from linker evaluation, not C allocation.

3. Sections are not segments

Sections organize linking and analysis: `.text`, `.rodata`, `.bss`, symbol tables. Program headers describe loadable segments for a loader. A section listing can show where the Multiboot2 header resides; a program-header listing shows what the ELF asks a loader to map or copy.

Use `readelf -h`, `-S`, and `-l` for distinct evidence:

- `-h`: class, machine, file type, and entry address;
- `-S`: section names, addresses, offsets, flags, and sizes;
- `-l`: loader-facing segments and section-to-segment mapping.

Do not infer file bytes for `.bss` merely from its in-memory size.

4. The Multiboot2 header arithmetic

The assembly defines:

```
magic + architecture + header_length + checksum ≡ 0 (mod 2^32)
```

Architecture value zero denotes the Multiboot2 i386 protected-mode entry contract. That is why GRUB enters this image in 32-bit mode even though most of the kernel is x86-64. The terminating header tag has type zero and size eight.

At entry, this project records:

- `EAX`: Multiboot2 bootloader magic;
- `EBX`: physical address of the Multiboot2 information structure.

Those are protocol facts. Saving them before reusing registers is a bootstrap implementation decision.

5. GRUB's role and its limit

`grub.cfg` selects:

```
multiboot2 /boot/toyos.elf
boot
```

GRUB locates the header, loads the ELF, prepares Multiboot2 information, and transfers control. It does not install ToyOS's final GDT, IDT, TSS, exception stubs, or serial driver. Those remain kernel work.

The Makefile copies the ELF and configuration into an ISO staging tree and runs `grub2-mkrescue`. Seeing files in `build/week6/iso/` proves staging; successfully booting the ISO proves more.

6. Relocations and symbols

Compilation creates references whose final addresses may be unknown. The linker resolves them under `linker.ld`. The map file `build/week6/toyos.map` records the selected layout and is useful when a debugger is unavailable.

`nm -n` orders symbols by address. `objdump -d` decodes executable bytes according to the ELF machine type. Neither proves that an instruction executed. Dynamic serial or debugger evidence is needed for that claim.

7. Practical challenge

Use `challenges/day-002-elf-boot-contract.sh` from the repository root.

Purpose: connect the linker and boot source to the generated ELF and ISO.

Predict: predict the ELF class, machine, entry symbol, lowest important section, and whether `.bss` occupies file bytes like `.text`.

Run: the wrapper builds Week 6, validates the Multiboot2 header, prints focused ELF metadata and symbols, and displays the packaged GRUB configuration.

Evidence: preserve the exact ELF machine, entry, `.multiboot2` location, load segments, `_start`, `__kernel_start`, `__kernel_end`, and validation result. Explain which tool interpreted each fact.

8. No claim beyond evidence

`grub2-file --is-x86-multiboot2` proves that the inspected file satisfies that tool's recognition test. It does not prove firmware compatibility, successful ISO boot, correct page tables, or correct exception handling.

A QEMU pass marker proves the tested image reached that marker under the recorded emulator configuration. It does not prove behavior on every physical machine.

Mastery: Explain · Draw · Observe · Build

Explain

- Why does the ELF entry point differ from C's `kernel_main`?
- Why must the Multiboot2 header be retained and early enough in the image?
- Why are sections and loadable segments not interchangeable?
- Why can a correct ELF still fail after transfer to `_start`?

Draw

Draw one diagram with file offsets on the left and linked memory addresses on the right. Place the Multiboot2 header, text, read-only data, data, and BSS. Mark which facts are consumed by the linker, GRUB, and CPU.

Observe

Compare `readelf` output with `linker.ld`. For each alignment boundary, identify whether the evidence is stated by source, produced by the linker, or observed in the final ELF.

Build

Create a compact boot-chain ledger:

```
claim | source/artifact | inspecting tool | observed value | limitation
```

Do not list a command without its question and expected evidence.

Current references

For exact protocol rules consult the current Multiboot2 specification and current GNU binutils documentation. ELF details should be checked against the System V ABI and AMD64 supplement applicable to the toolchain. Local notes are helpful orientation, not replacements for those versioned documents.

Now check your understanding

You have finished the theory for this section. Before moving on, use the quick check below to find anything that still feels uncertain, then reinforce the idea with the practical lab when one is provided.

Quick theory check

Answer from memory first. Explanations appear only after you submit your choices.

- Start the interactive theory check
- View quiz source on GitHub

Put it into practice — Elf Boot Contract

Format: shell lab

What you will do: Inspect the linked ELF, Multiboot2 recognition, and packaged GRUB contract.

Predict the important result first, then run the lab and explain any difference between prediction and observation.

- Open the lab file

Copy and run:

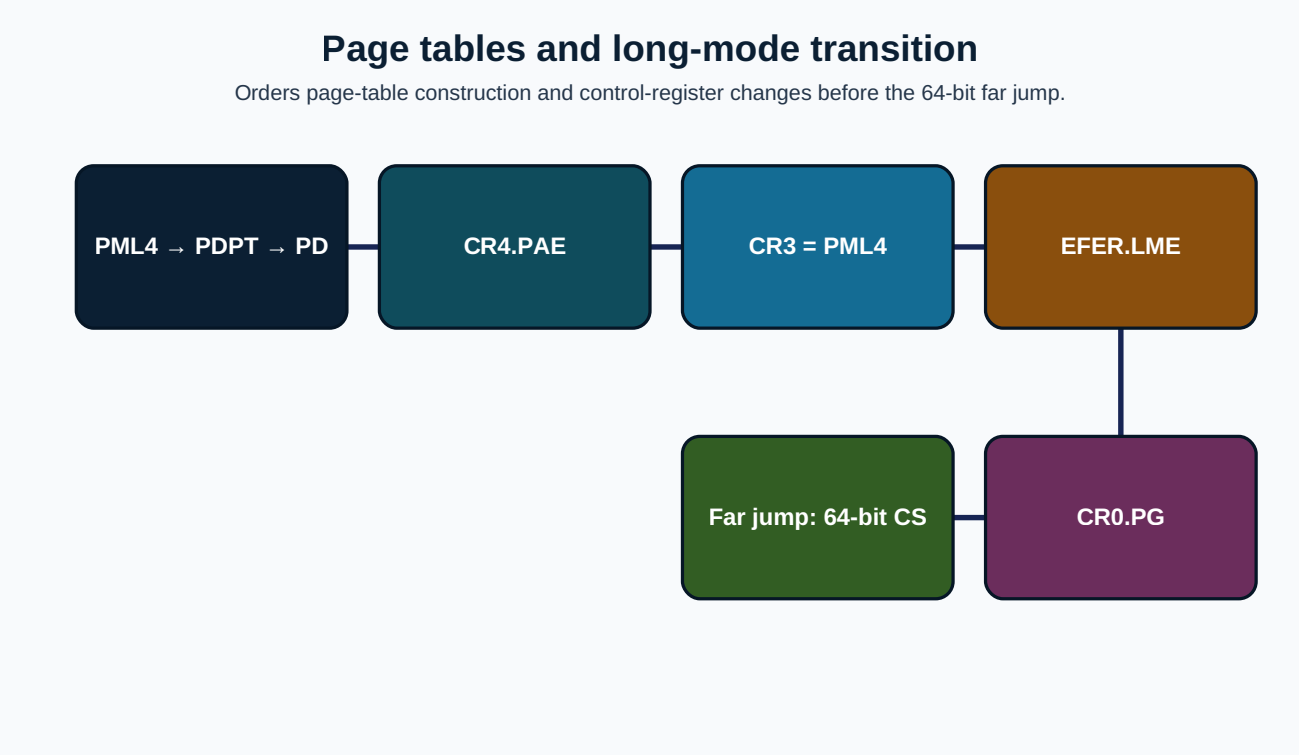
```
bash 06-toyos-bootstrap-and-exceptions/challenges/day-002-elf-boot-contract.sh
```

- View lab source on GitHub

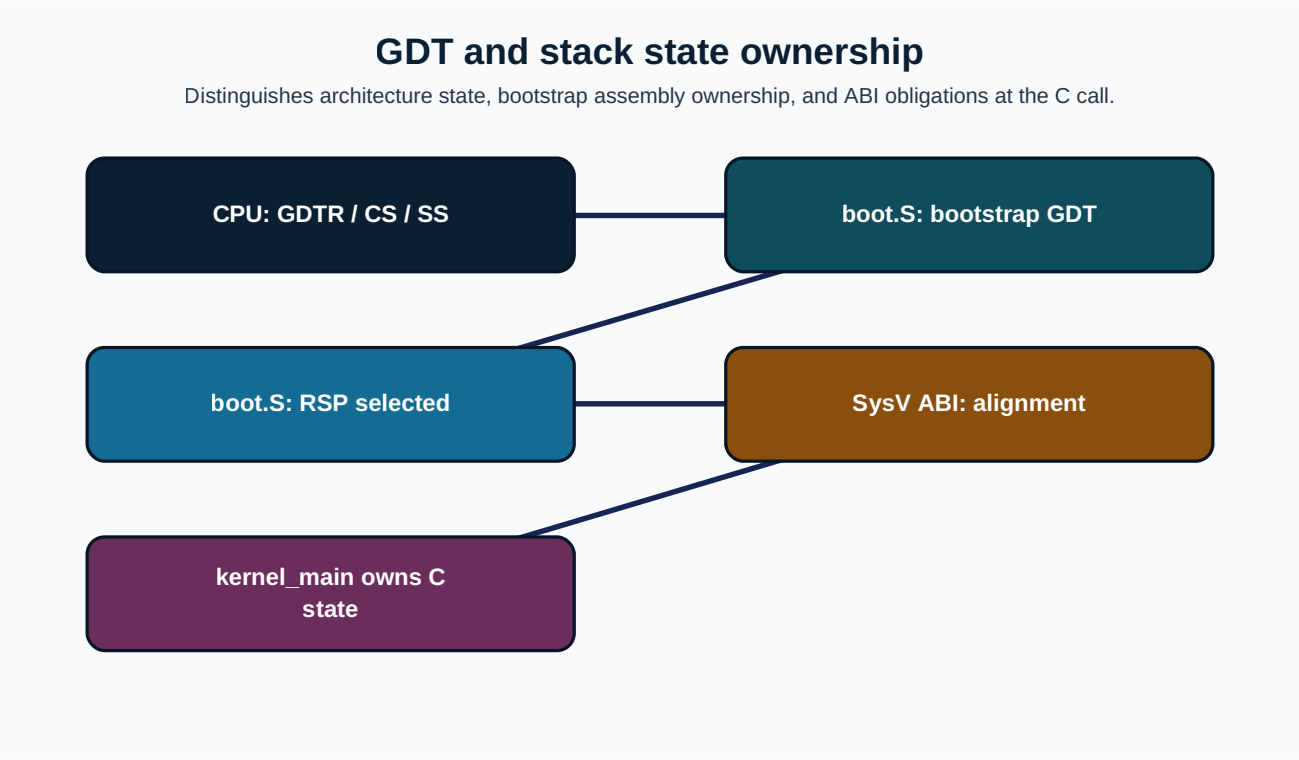
Day 3 — Long Mode, Initial Pages, GDT, and Stack

Target time: approximately 3 hours

> GRUB enters a 32-bit contract, but GCC emitted 64-bit kernel code. Which state transitions make the first C call legal?



Orders page-table construction and control-register changes before the 64-bit far jump.



Distinguishes architecture state, bootstrap assembly ownership, and ABI obligations at the C call.

The transition in `projects/toyos/src/boot.S` is short because it assumes a specific starting contract and creates only the state this milestone needs. Read it as a dependency graph, not as magic register incantations.

---

1. Preconditions and ordering

At `_start`, ToyOS immediately disables maskable interrupts, saves `EAX` and `EBX`, and selects a bootstrap stack. Disabling interrupts does not disable exceptions or NMI. It avoids ordinary interrupt delivery before an IDT exists.

The required chain is:

```
build page tables
  → set CR4.PAE
  → load CR3 with PML4 address
  → set EFER.LME
  → set CR0.PG
  → load bootstrap GDT
  → far jump to a 64-bit code descriptor
```

Changing this order can make the transition architecturally invalid. Consult the current Intel or AMD architecture manual for exact prerequisites; source order is evidence of ToyOS's sequence, not the normative rule by itself.

2. The initial translation

ToyOS allocates one page each for a PML4, PDPT, and page directory. It links entry zero at each upper level, then fills 512 page-directory entries. Each entry maps one 2 MiB page with flags `0x83`:

```
present | writable | page-size
```

For index `i`, the physical base is `i << 21`. The result identity maps the first 1 GiB:

```
virtual 0x00000000..0x3fffffff
    == physical 0x00000000..0x3fffffff
```

This supports the 1 MiB-linked kernel, bootstrap structures, stack, and Multiboot information only if GRUB placed them within that range. The project README states this deliberate assumption. Identity mapping is implementation policy; long mode does not mandate it.

The assembly writes only the low 32 bits of the upper-level addresses and zeros the high half of each 64-bit PDE. This relies on all bootstrap objects being below 4 GiB, which the current link placement satisfies.

3. GDT still matters in long mode

Long mode reduces ordinary segmentation, but a valid code descriptor and selector remain necessary. The bootstrap GDT contains null, code, and data descriptors. `lgdt` loads its pointer; a far jump to selector `0x08` both changes `CS` and reaches `long_mode_entry`.

Inside 64-bit code, ToyOS loads data selectors, clears `FS` and `GS` selectors, establishes `RSP`, and clears `RBP`. Later, `arch.c` installs the runtime GDT with a TSS descriptor. Bootstrap GDT and runtime GDT serve different phases.

4. Stack and ABI handoff

The BSS reserves a 32 KiB stack aligned to 16 bytes. The stack grows toward lower addresses. Immediately before `call kernel_main`, `RSP` points at `boot_stack_top`. The call pushes an eight-byte return address.

System V AMD64 stack alignment rules concern the stack at call boundaries and function entry. Check generated disassembly when making a precise alignment claim; do not infer all compiler behavior from source.

The assembly moves saved boot values into `EDI` and `ESI`. Writing a 32-bit subregister zeros the upper half, matching the `uint32_t` C parameters. This is the concrete ABI bridge.

If `kernel_main` returns, the bootstrap enters a `cli; hlt` loop. Normal Week 6 completion instead calls the implementation's QEMU exit device.

5. What architecture supplies and ToyOS chooses

Architecture:

- meanings of `CR0.PG`, `CR4.PAE`, `CR3`, and `EFER.LME`;
- four-level translation in the selected x86-64 mode;

- descriptor loading and far control transfer behavior;
- instruction encodings and exception conditions.

ABI:

- integer argument registers;
- stack alignment and register preservation conventions;
- object and symbol conventions used by C and assembly.

ToyOS implementation:

- first-GiB identity map;
- 2 MiB pages;
- stack size and location;
- selectors `0x08` and `0x10`;
- choice to enter a C function with two Multiboot values.

6. Practical challenge

Use `challenges/day-003-long-mode-trace.sh` from the repository root.

Purpose: correlate bootstrap source, linked addresses, decoded instructions, and the normal boot marker.

Predict: calculate the virtual/physical interval covered by 512 entries of 2 MiB each. Predict which registers carry `kernel_main` arguments and the serial marker expected after `arch_init`.

Run: the wrapper builds the canonical image, prints the `_start` through `long_mode_entry` disassembly and relevant symbols, then runs the existing normal Week 6 regression.

Evidence: annotate instructions that establish each dependency. Preserve the `BOOT: Multiboot2 -> 32-bit entry -> long mode PASS` marker and explain that it is emitted after `arch_init`, not at the instant long mode begins.

7. Architecture lens at the mode boundary

This transition is specifically x86-64.

- AArch64 has exception levels and configured execution states rather than x86's protected-mode-to-long-mode sequence. Firmware handoff and page-table enablement use different system registers and barriers.
- RV64 uses privilege modes and `satp`-selected translation schemes. There is no GDT or segment selector; trap vectors and return state use CSRs.

The portable idea is narrower: assembly establishes architectural state and an ABI-compliant stack before calling C. The registers and ordering are architecture-specific.

Mastery: Explain · Draw · Observe · Build

Explain

1. Why is a far jump needed after enabling long mode?

2. Why does identity mapping simplify this bootstrap?
3. Why does a 1 GiB initial map imply a placement assumption?
4. Why is the initial stack simultaneously an implementation object and an ABI resource?

Draw

Draw the three allocated table pages and one translation for address `0x00100000`. Show indices, the 2 MiB leaf, and unchanged offset. Then mark where `CR3` points.

Observe

Match each control-register update in disassembly to its source line. Record whether your tool prints symbolic names or only opcodes. Tool formatting is not architectural evidence.

Build

Write a transition checklist where each row has:

```
precondition | instruction | resulting invariant | later consumer | failure
```

Include the page tables, mode bits, GDT, stack, and argument registers.

Evidence limits

Disassembly proves bytes exist at linked addresses. The serial regression proves this QEMU run traversed enough of the path to initialize architecture and print. Neither alone proves the path on all x86-64 implementations.

Now check your understanding

You have finished the theory for this section. Before moving on, use the quick check below to find anything that still feels uncertain, then reinforce the idea with the practical lab when one is provided.

Quick theory check

Answer from memory first. Explanations appear only after you submit your choices.

- Start the interactive theory check
- View quiz source on GitHub

Put it into practice — Long Mode Trace

Format: shell lab

What you will do: Correlate the 32-bit bootstrap, long-mode entry, symbols, and boot regression.

Predict the important result first, then run the lab and explain any difference between prediction and observation.

- Open the lab file

Copy and run:

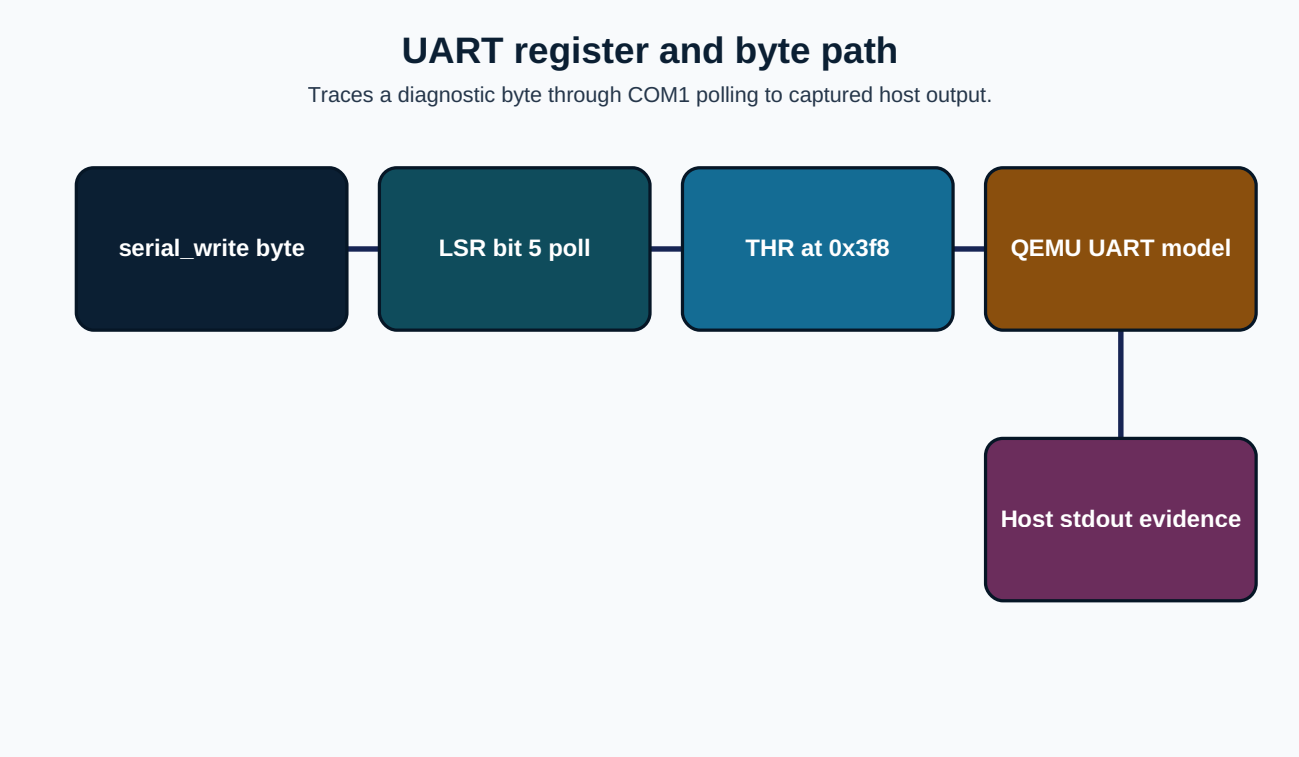
```
bash 06-toyos-bootstrap-and-exceptions/challenges/day-003-long-mode-trace.sh
```

- View lab source on GitHub

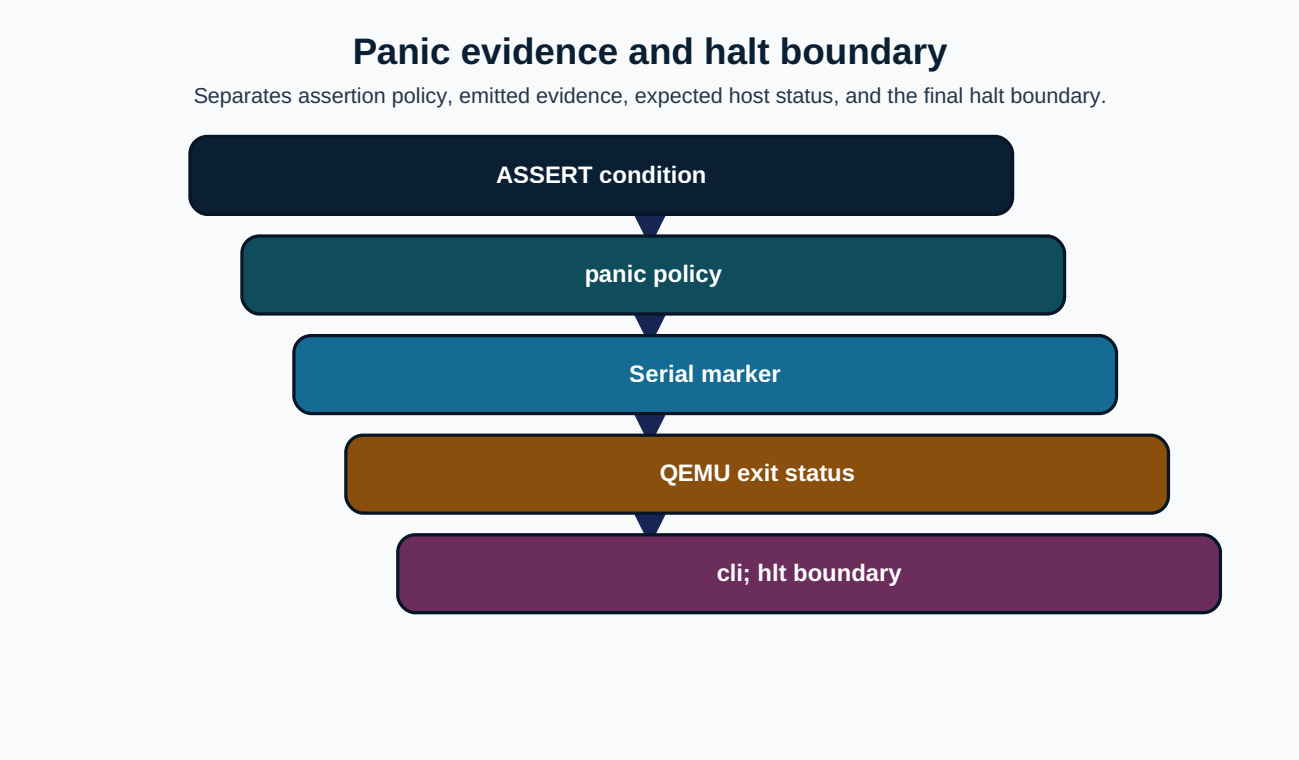
Day 4 — COM1 Serial, Panic, and Assert

Target time: approximately 2½–3 hours

> When the screen and filesystem do not exist, what evidence can the kernel emit before it deliberately stops?



Traces a diagnostic byte through COM1 polling to captured host output.



Separates assertion policy, emitted evidence, expected host status, and the final halt boundary.

ToyOS uses the first 16550-compatible serial port, conventionally COM1 at I/O base `0x3f8`. QEMU connects that byte stream to the host process's standard output. This is both an early diagnostic path and the observable channel for regression tests.

---

1. Initializing the UART

`serial_init` performs programmed I/O with `outb`. It:

1. disables UART interrupts;
2. enables divisor-latch access;
3. writes divisor 1;
4. selects eight data bits, no parity, one stop bit;
5. configures and clears FIFOs;
6. sets modem-control outputs.

Those writes target a 16550-style interface implemented by QEMU in this test environment. The x86 `in`/`out` instructions and I/O privilege behavior are architecture/platform mechanisms. The selected baud divisor and polling driver are implementation choices.

Exact physical serial timing is not observed when QEMU routes the emulated device to a host stream. The useful evidence is ordered bytes from the emulated guest.

2. Polling and liveness

`serial_putc` waits until line-status bit `0x20` says the transmit holding register is empty, then writes one byte. Polling keeps the early path small: no IRQ controller, receive path, queue, or scheduler is required.

The trade-off is liveness. If the modeled device never becomes ready, the kernel spins forever. There is no timeout in this primitive. The outer test harness supplies a 12-second host timeout so automation cannot hang indefinitely.

`serial_write` inserts carriage return before newline. That is terminal presentation policy, not a C or x86 requirement.

3. Formatting without libc

`serial_hex` emits exactly sixteen hexadecimal digits after `0x`, exposing the full 64-bit value. `serial_dec` uses a fixed 21-byte local array and constructs digits backward.

These routines avoid dynamic allocation and variadic formatting. Their bounded behavior is easier to reason about while exception handling may already be under stress. They are not general `printf` replacements.

4. Panic is policy

`panic_at` prints:

```
PANIC: <message> at <file>:<line>
```

It then writes a code to QEMU's `isa-debug-exit` device. The CPU does not define “panic.” It defines instructions, exceptions, and halt behavior; ToyOS decides that an unrecoverable invariant violation should report and terminate this test VM.

The Makefile configures the device at its default I/O port. QEMU reports a host status derived from the guest value. The harness expects status 33 for normal completion, 35 for expected architecture faults, and 37 for the expected assertion profile. These numbers are test-protocol evidence, not portable process exit semantics for kernels.

5. Assertion versus architectural exception

`ASSERT(false)` expands to a normal C branch and call to `panic_at`. It is not an x86 exception. By contrast, division by zero and an unmapped write cause processor exceptions that enter through the IDT.

This distinction matters in a report:

```
assertion failure:
  C condition → ToyOS macro → panic_at → serial → debug-exit

page fault:
  memory instruction → CPU translation failure → IDT/stub/frame
  → C handler → serial → debug-exit
```

Both stop this milestone, but their entry mechanisms and evidence differ.

6. Regression harness as evidence

`scripts/check-qemu.sh` creates a temporary host log, launches QEMU under `timeout`, captures status, prints the log, checks the expected status, and searches for an exact serial marker. It removes the temporary file on exit.

The two-part check matters. A marker without the expected exit might precede a later hang. An exit status without the marker might come from the wrong path. Together they identify the tested success protocol more narrowly.

The harness still cannot prove every serial byte, ordering under concurrency, or behavior on physical UART hardware.

7. Practical challenge

Use `challenges/day-004-serial-panic-assert.sh` from the repository root.

Purpose: compare normal completion with the deliberate assertion path.

Predict: list the serial lines common to both runs, the first line that diverges, and each expected host status.

Run: the wrapper invokes the existing normal Week 6 check and expected assert profile. It also prints the serial/panic source region so output can be correlated with implementation.

Evidence: preserve `WEEK6: PASS`, `INJECT: assertion failure`, `PANIC: assertion failed: false`, `EXPECTED-FAULT: assert PASS`, and both log-check markers. The panic line's exact source line can change when the canonical file changes; report the observed value rather than memorizing it.

8. Debugging discipline

Early logs should be:

- deterministic enough for exact checks;
- explicit about milestone and event;
- usable before allocation and interrupts;
- bounded where possible;
- honest about whether a result is pass, expected fault, or panic.

A log statement is evidence that execution reached the write. It is not proof that every earlier subsystem is fully correct. Prefer several independent checks over one celebratory banner.

Mastery: Explain · Observe · Build

Explain

- Why does polling reduce dependencies but risk hanging?
- Why is `ASSERT` not an architectural exception?
- Why does a kernel test need both a serial marker and host status?
- Why does emulated COM1 output not measure real UART timing?

Observe

Create a two-column trace for normal and assertion profiles. Mark the last shared event, first divergent event, termination mechanism, and host-visible evidence.

Build

Design, on paper, one additional bounded diagnostic such as printing `RSP`. Specify where it is safe, what it would prove, and what it would not prove. Do not edit the canonical project for this challenge.

Optional references

Use current 16550-compatible UART documentation and QEMU's `isa-debug-exit` documentation for exact device behavior. The current Intel or AMD manuals define port-I/O instructions. Local kernel-debugging examples may illustrate panic policy, but are not specifications.

Now check your understanding

You have finished the theory for this section. Before moving on, use the quick check below to find anything that still feels uncertain, then reinforce the idea with the practical lab when one is provided.

Quick theory check

Answer from memory first. Explanations appear only after you submit your choices.

- Start the interactive theory check
- View quiz source on GitHub

Put it into practice — Serial Panic Assert

Format: shell lab

What you will do: Compare ToyOS normal serial completion with its deliberate assertion panic.

Predict the important result first, then run the lab and explain any difference between prediction and observation.

- Open the lab file

Copy and run:

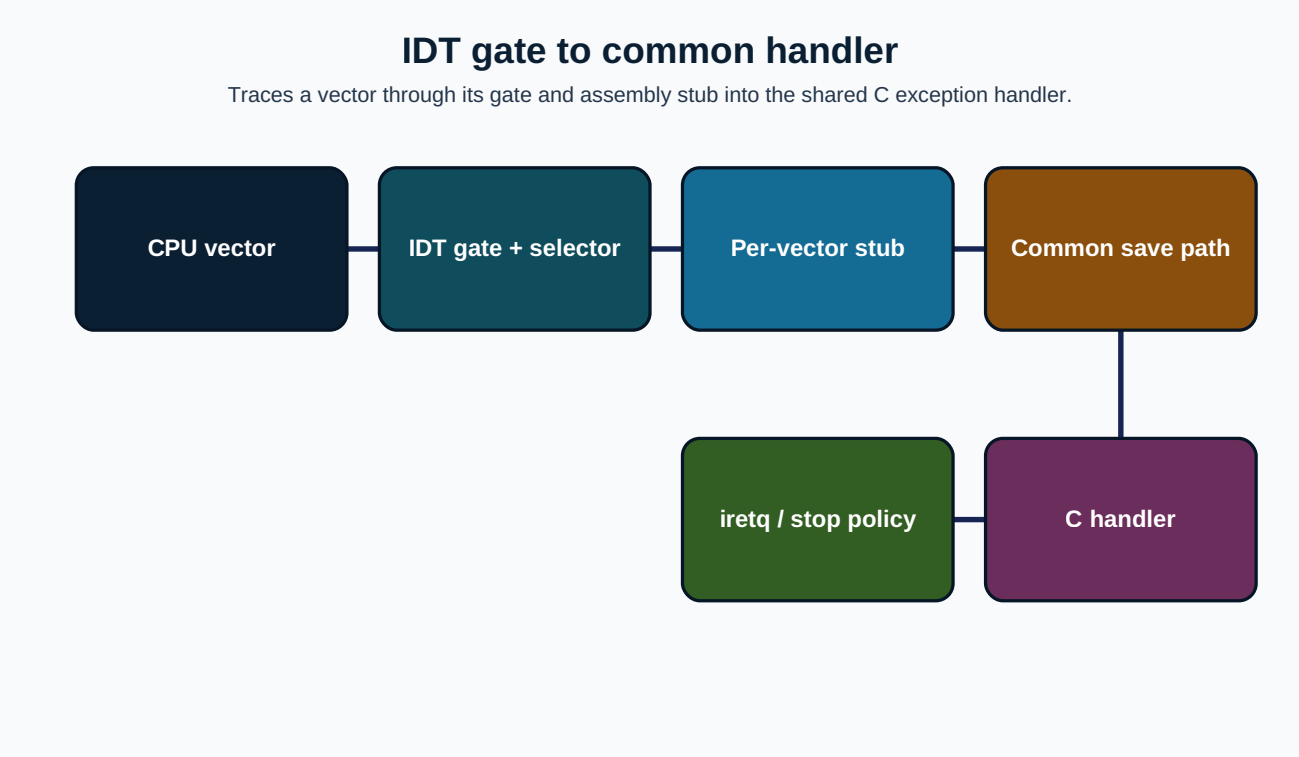
```
bash 06-toyos-bootstrap-and-exceptions/challenges/day-004-serial-panic-assert.sh
```

- View lab source on GitHub

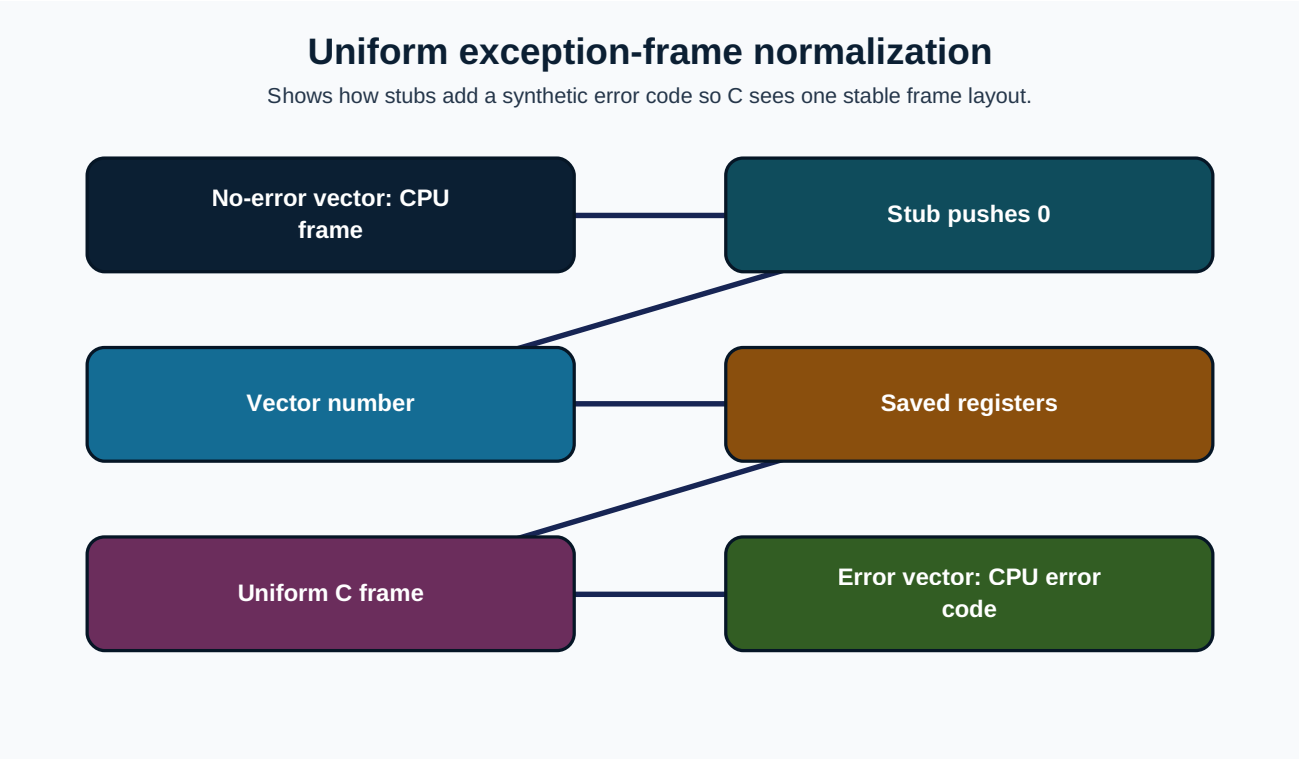
Day 5 — IDT, Exception Stubs, and a Uniform Saved Frame

Target time: approximately 3 hours

> x86 exceptions do not all push the same fields. How can one C handler receive one predictable structure without losing the hardware's meaning?



Traces a vector through its gate and assembly stub into the shared C exception handler.



Shows how stubs add a synthetic error code so C sees one stable frame layout.

ToyOS combines architecture-created state with assembly-created state. The `struct interrupt_frame` is correct only while its fields match the exact stack layout produced by both.

---

1. From vector to handler

The runtime IDT has 256 entries. Week 6 installs gates for vectors 0–31 and also prepares vector 32 for Week 7's timer. Each entry includes:

- low/middle/high pieces of the handler address;
- kernel code selector `0x08`;
- an IST index;
- attributes `0x8e`;
- a reserved zero field.

`0x8e` describes a present ring-0 interrupt gate. Gate encoding is x86-64 architecture. The chosen handlers, selector, and IST assignment are ToyOS.

`lidt` installs a descriptor containing the IDT base and byte limit. Building an array is insufficient; the architecture consults the table selected by `IDTR`.

2. Hardware entry is not the C ABI

On exception entry without a privilege-level change, the processor supplies at least saved `RIP`, `CS`, and `RFLAGS`. Some vectors also push an error code. A privilege or stack transition changes the frame further. Consult current architecture manuals for exact vector-specific behavior.

C compilers do not automatically understand that raw frame. An ordinary C function expects its platform ABI. Therefore each assembly stub normalizes the stack and `isr_common` saves general registers before calling C.

3. Normalizing error-code differences

For a vector without a CPU error code, `ISR_NOERR` pushes:

```
synthetic error = 0
vector number
```

For a vector where the CPU already pushed an error code, `ISR_ERR` pushes only the vector number. In both cases, the top of the pre-register frame becomes:

```
vector, error, rip, cs, rflags
```

This is a normalization layer. The synthetic zero must never be mistaken for proof that hardware supplied an error code of zero.

Maintaining the correct vector list is critical. If a stub wrongly adds or omits a word, every later field shifts and `iretq` consumes corrupt state.

4. Push order versus structure order

Stacks grow downward. `isr_common` pushes `RAX`, `RBX`, ..., ending with `R15`. After the final push, `RSP` points at `R15`. The C structure therefore begins:

```
uint64_t r15, r14, r13, ... , rbx, rax;
uint64_t vector, error, rip, cs, rflags;
```

The apparent reverse order is deliberate. `movq %rsp, %rdi` passes a pointer to this frame as the first System V AMD64 argument.

`cld` clears the direction flag before C. The ABI expects forward string direction. This is an ABI hygiene step performed at an architecture entry boundary.

5. Return and register ownership

After `interrupt_handler` returns, assembly pops registers in reverse, discards vector and error with `addq $16, %rsp`, then executes `iretq`.

The breakpoint vector is intentionally recoverable: the handler prints a self-test marker and returns. Divide and page expected-fault profiles exit from the C handler instead. Unexpected exceptions panic.

This kernel always saves its listed general registers, rather than relying on ordinary caller-saved/callee-saved rules across asynchronous entry. Floating-point/vector state is outside this milestone, reinforced by `-mgeneral-regs-only`.

6. Precise exceptions and saved RIP

A useful exception frame identifies the interrupted architectural context. The meaning of saved `RIP` depends on the exception class: for a fault it commonly identifies the instruction to retry; trap semantics can identify a subsequent point. Do not state one universal rule for every vector.

ToyOS's divide-error and page-fault tests do not resume, so they verify entry, decode, and diagnostics rather than restart correctness. The `int3` breakpoint does return and provides bounded evidence for `iretq`.

7. Practical challenge

Use `challenges/day-005-exception-frame.sh` from the repository root.

Purpose: verify the implementation's frame agreement using static layout inspection plus dynamic breakpoint and divide-error paths.

Predict: calculate byte offsets for `r15`, `rax`, `vector`, `error`, and `rip`. Predict why vector 0 needs a synthetic error word and what markers the normal and divide profiles should emit.

Run: the wrapper prints the C structure and assembly common entry, checks the offsets with a temporary hosted helper that includes the canonical header, then runs normal and divide expected-fault tests.

Evidence: preserve measured offsets, `SELFTEST: IDT breakpoint frame PASS`, and `EXPECTED-FAULT: divide PASS`. The hosted `offsetof` check proves the compiler's C layout under those include/type rules; the dynamic tests connect that layout to selected guest paths.

8. Architecture lens at exception entry

The need to bridge raw architectural state to a software frame is common; the shape is not.

- x86-64 selects an IDT gate and may push an error code; software often adds a vector and register save area before `iretq`.
- AArch64 vectors branch through `VBAR_ELx`; exception state is held in registers such as `ELR_ELx` and `SPSR_ELx`, which entry code saves into a software frame before `eret`.
- RISC-V records cause, return PC, and status in CSRs such as `scause`, `sepc`, and `sstatus`; a trap vector saves general registers before `sret`.

Do not project x86's hardware stack layout or error-code convention onto the other architectures.

Mastery: Explain · Draw · Observe · Build

Explain

1. Why do no-error and error-code vectors use different stub macros?
2. Why does the C structure list registers opposite the push sequence?
3. Why must `cld` run before calling C?
4. Why does the breakpoint test exercise more than a log call?

Draw

Draw the stack from high to low addresses at C-handler entry for vector 0 and vector 14. Color hardware-pushed, stub-pushed, and common-entry-pushed words. Both drawings must match one structure from `RSP`.

Observe

Correlate `isr_stub_table[0]`, IDT vector zero, `isr0`, `isr_common`, and `interrupt_handler`. At each edge, identify the address or calling contract.

Build

Write three invariants that an automated frame test should check. Include one offset, one vector/error relationship, and one successful return path.

Evidence limits

Passing breakpoint and divide tests exercises two vectors. It does not prove all 32 stubs, privilege transitions, nested exceptions, NMI behavior, or floating-point state preservation.

Now check your understanding

You have finished the theory for this section. Before moving on, use the quick check below to find anything that still feels uncertain, then reinforce the idea with the practical lab when one is provided.

Quick theory check

Answer from memory first. Explanations appear only after you submit your choices.

- Start the interactive theory check
- View quiz source on GitHub

Put it into practice — Exception Frame

Format: shell lab

What you will do: Check C frame offsets and exercise breakpoint and divide exception paths.

Predict the important result first, then run the lab and explain any difference between prediction and observation.

- Open the lab file

Copy and run:

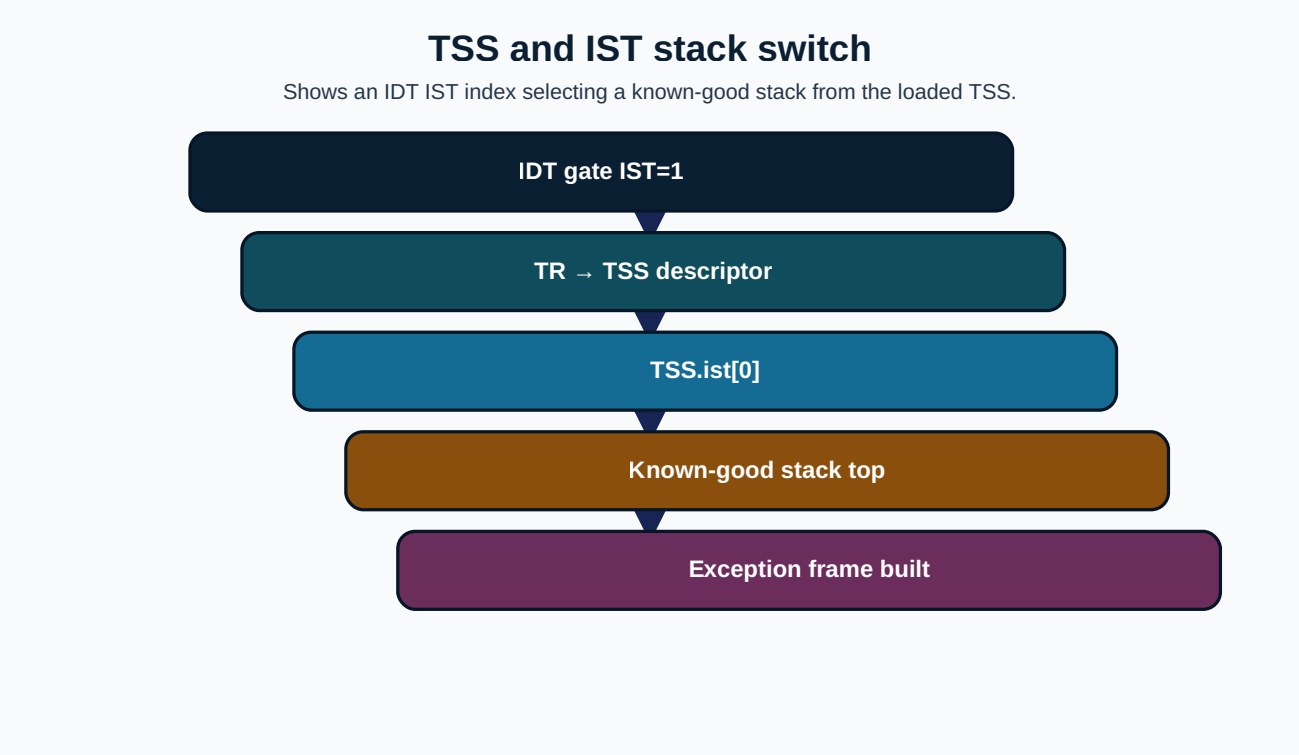
```
bash 06-toyos-bootstrap-and-exceptions/challenges/day-005-exception-frame.sh
```

- View lab source on GitHub

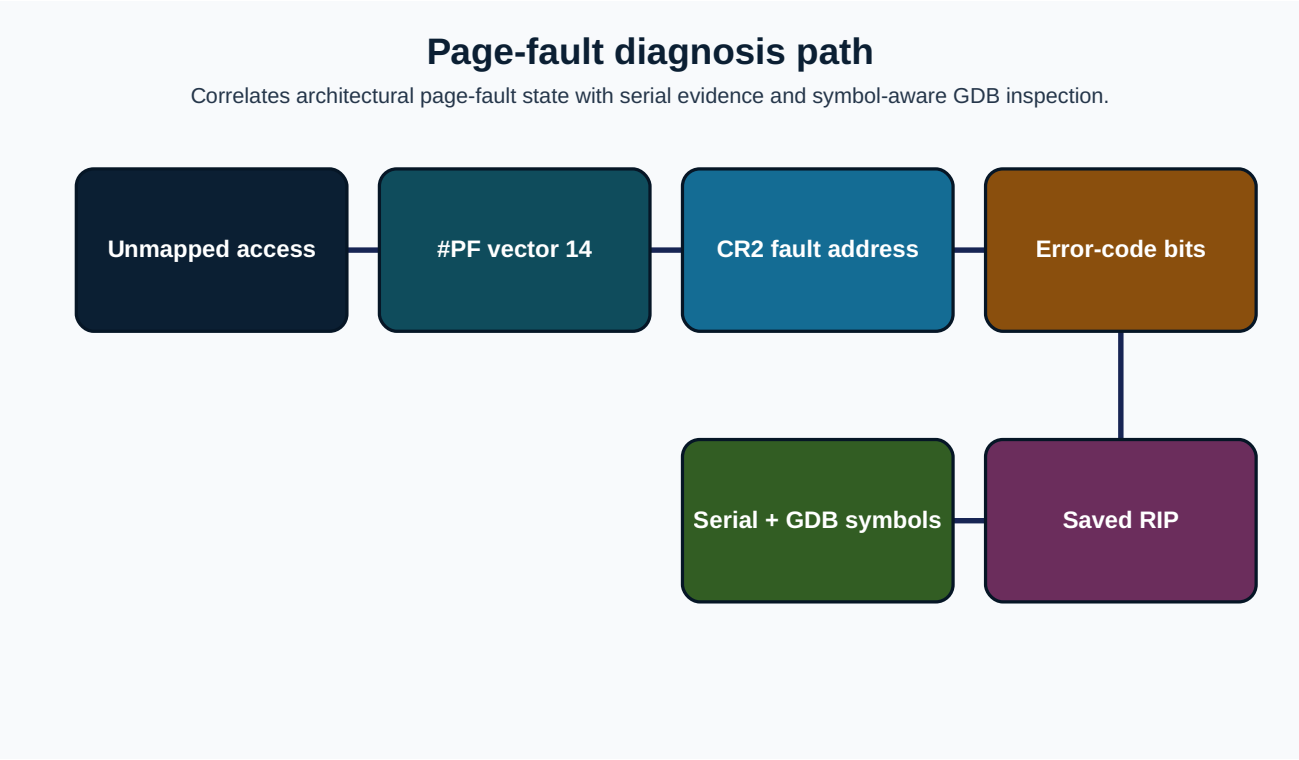
Day 6 — TSS, IST, Page Faults, and GDB Diagnosis

Target time: approximately 3 hours

> If the current stack contributed to the fault, should the fault handler trust it?



Shows an IDT IST index selecting a known-good stack from the loaded TSS.



Correlates architectural page-fault state with serial evidence and symbol-aware GDB inspection.

The x86-64 Task State Segment and Interrupt Stack Table let selected IDT gates switch to a known stack. ToyOS uses IST entry 1 for NMI, double fault, and page fault, then exposes enough state to diagnose a deliberate unmapped write.

---

1. Runtime GDT and TSS

`arch.c` creates a five-quadword GDT: null, kernel code, kernel data, and a two-slot 64-bit TSS descriptor. Unlike the bootstrap GDT, this table supports loading `TR` with selector `0x18`.

The TSS stores:

- privilege stack pointers `rsp[0..2]`;
- seven IST pointers;
- an I/O-map offset.

ToyOS sets `rsp[0]` and `ist[0]` to the top of one aligned 16 KiB `interrupt_stack`. It sets `iomap_base` beyond the TSS body and executes `ltr`.

This does not implement hardware task switching. In 64-bit mode, ToyOS uses the TSS for stack-selection metadata.

2. What IST buys—and what this implementation shares

An IDT gate's low IST bits select one of the TSS IST pointers. ToyOS selects IST index 1 for vectors 2, 8, and 14:

```
NMI, double fault, page fault → interrupt_stack
```

A known stack can preserve diagnostic ability if the interrupted `RSP` is unusable. However, this implementation shares one IST stack among three vectors and is single-core. It does not provide per-CPU stacks, guard pages, or robust nesting policy.

Double fault is vector 8. It is not simply “two exceptions happened.” Exact classification depends on combinations of exceptions during delivery; use the current architecture manual's exception-combination rules. A failure during double-fault delivery can lead to shutdown/triple-fault behavior, which ToyOS does not test here.

3. Deliberate page fault

The expected page profile writes to linear address `0x70000000`. Bootstrap page tables identity map only the first 1 GiB, ending at `0x3fffffff`. Therefore the chosen address lies outside the initial mapping.

The C expression uses a `volatile` lvalue so the compiler must emit the observable access under its abstract-machine rules. The invalid dereference would be undefined behavior in general C reasoning; this controlled kernel test relies on inspected code generation and the configured x86 translation state to provoke vector 14.

4. Decoding page-fault evidence

On vector 14, ToyOS reads `CR2` for the faulting linear address and decodes low error-code bits:

- bit 0: protection violation versus non-present;
- bit 1: write versus read;
- bit 2: user versus supervisor;
- bit 3: reserved paging-structure bit;
- bit 4: instruction fetch.

For this test, predict:

```
address=0x0000000070000000
not-present write supervisor
```

Treat additional error bits and feature-dependent semantics according to the current Intel or AMD manual. ToyOS prints only this bounded subset.

`CR2` supplies the address; saved `RIP` identifies the instruction context; the error code classifies the attempted access. None alone is a complete diagnosis.

5. Symbolized static diagnosis

The linked ELF retains debug information and symbols because the build uses `-g3`. Even without a live VM, use:

```
symbol address/range → decoded instruction → source line
```

`nm -n`, `objdump -dS`, and `addr2line` can correlate the injected path. Optimized code complicates one-to-one source mapping: functions can inline, statements can combine, and one source line can span several instructions. Report the observed mapping rather than claiming every instruction uniquely belongs to one C statement.

6. Live GDB workflow

The project provides two coordinated targets:

1. `make debug-fault-week6-page` builds the fault image and starts QEMU paused with a GDB stub on TCP port 1234.
2. `make gdb-fault` loads `build/week6-page/toyos.elf`, connects, breaks at `interrupt_handler`, and continues.

At the breakpoint, useful questions are:

- What are `frame->vector`, `frame->error`, and `frame->rip`?
- Does `$cr2` equal the injected address?
- What instruction is at `frame->rip`?
- Which source line and symbol contain it?
- Which stack contains `frame`, and is it within `interrupt_stack`?

The challenge wrapper does not start a persistent debug server. It prints a purpose-labelled command guide and performs deterministic static checks. Use the two-terminal live workflow only when you can supervise and stop QEMU.

7. Practical challenge

Use `challenges/day-006-page-fault-diagnosis.sh` from the repository root.

Purpose: produce repeatable page-fault evidence and prepare a symbol-aware live diagnosis without leaving a paused VM behind.

Predict: write the vector, CR2 value, decoded low error bits, expected host status, and likely source function before running.

Run: the wrapper invokes the canonical expected-page regression, locates relevant symbols/disassembly in the exact fault ELF, verifies GDB script paths, and prints the supervised two-terminal procedure.

Evidence: preserve the exception line, page-fault line, expected marker, symbol addresses, and source/disassembly correlation. If you perform the optional GDB session, record the ELF build path and exact frame values.

8. Architecture lens at fault reporting

At this real boundary, all three architectures report fault context differently:

- x86-64 uses vector 14, an error code, `CR2`, IDT entry, and optional IST.
- AArch64 reports syndrome in `ESR_ELx`, fault address in `FAR_ELx`, and return state in `ELR_ELx`/`SPSR_ELx`; vector choice depends on origin and stack.
- RISC-V reports cause in `scause`, address/detail in `stval`, and return PC in `sepc`, entering through `stvec`.

The shared debugging method is cause + address + saved PC + symbolized binary. The registers, encodings, and stack mechanisms are architectural.

Mastery: Explain · Observe · Build

Explain

- Why can IST preserve a diagnostic path after stack corruption?
- Why is sharing one IST stack a limitation?
- Why are CR2, error code, and RIP complementary?
- Why must GDB load the exact ELF used by QEMU?

Observe

Annotate the page-fault serial line field by field. For each field, state its architectural source and the ToyOS code that formats it.

Build

Write a fault worksheet containing:

```
image/profile | vector | error | address | saved PC | symbol/source
prediction | observation | interpretation | evidence limit
```

Complete it from actual output. Do not invent a live-debug value if you ran only the static challenge.

Evidence limits

The test establishes one non-present supervisor write under QEMU TCG. It does not validate protection faults, user faults, execute faults, nested faults, real hardware, or robust double-fault recovery.

Now check your understanding

You have finished the theory for this section. Before moving on, use the quick check below to find anything that still feels uncertain, then reinforce the idea with the practical lab when one is provided.

Quick theory check

Answer from memory first. Explanations appear only after you submit your choices.

- Start the interactive theory check
- View quiz source on GitHub

Put it into practice — Page Fault Diagnosis

Format: shell lab

What you will do: Reproduce the page fault and prepare exact-ELF symbolized GDB diagnosis.

Predict the important result first, then run the lab and explain any difference between prediction and observation.

- Open the lab file

Copy and run:

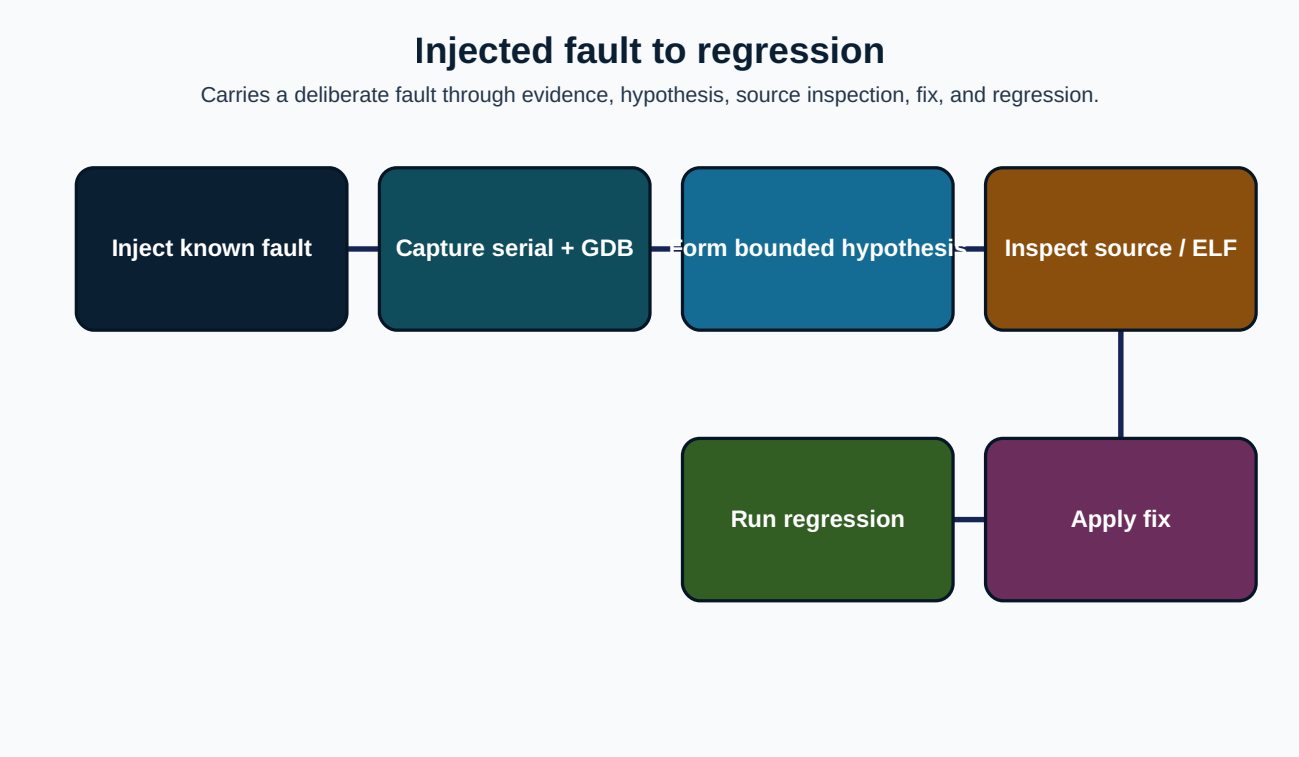
```
bash 06-toyos-bootstrap-and-exceptions/challenges/day-006-page-fault-diagnosis.sh
```

- View lab source on GitHub

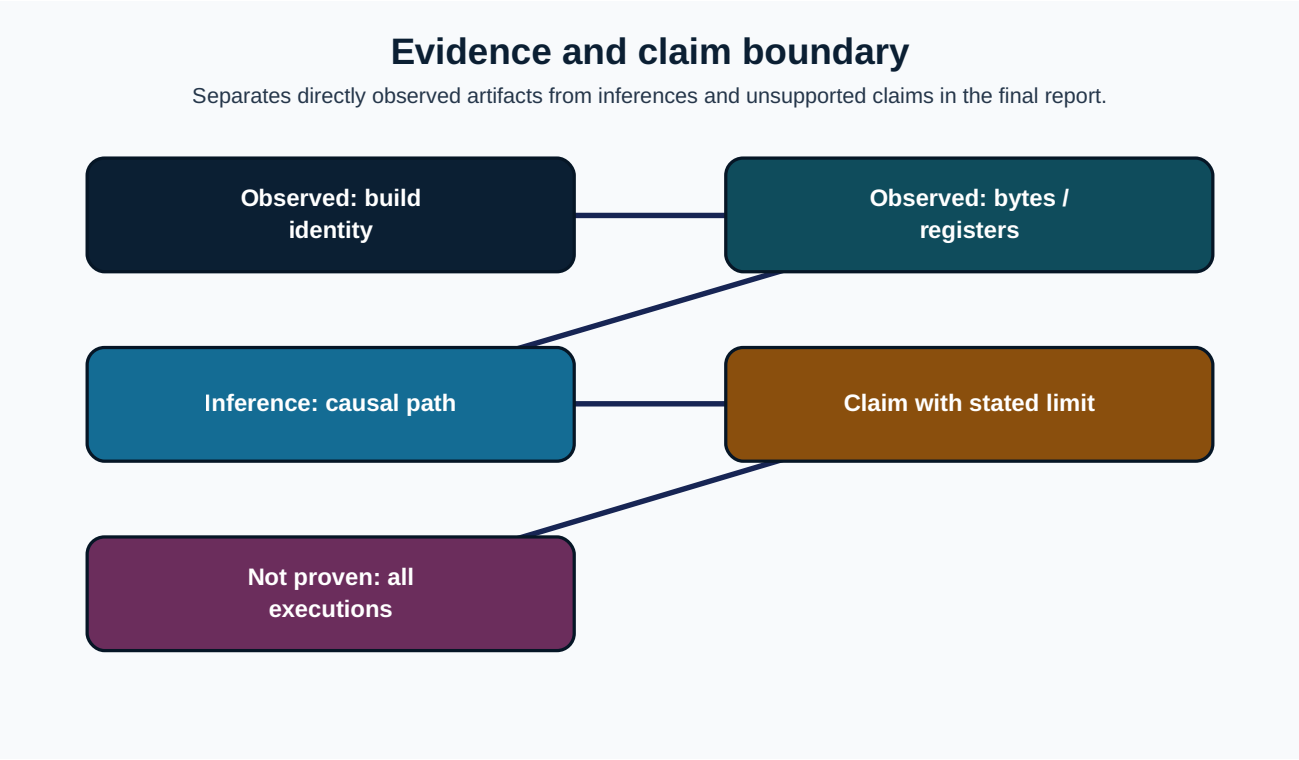
Day 7 — Integrated Fault-Debug Report and Systems Comparison

Target time: approximately 3 hours

> A fault report is useful only when another person can reproduce its evidence, distinguish fact from inference, and see where the model stops.



Carries a deliberate fault through evidence, hypothesis, source inspection, fix, and regression.



Separates directly observed artifacts from inferences and unsupported claims in the final report.

Today combines the Week 6 artifacts into one bounded report. The goal is not more kernel code. It is a defensible chain from C source to architectural fault state, exception entry, diagnostics, and symbolized location.

---

1. Start with an evidence question

Use the deliberate page fault as the main case:

> Why did the Week 6 page-fault profile stop, and what evidence identifies the faulting access and handler path?

Keep the assertion and divide profiles as controls. The assertion reaches panic without IDT exception entry. Divide uses exception vector 0 without a hardware error code. Together they help distinguish mechanisms.

2. Report structure

A strong report has seven parts:

1. **Artifact identity:** repository path, profile, ELF path, and tool versions relevant to interpretation.
2. **Prediction:** vector, address, access class, frame shape, marker, status.
3. **Static evidence:** source excerpts, linker map/symbols, disassembly, and exact debug ELF.
4. **Dynamic evidence:** serial lines, expected markers, QEMU statuses.

5. **Causal explanation:** each transition labelled C, ABI, architecture, or ToyOS implementation.
6. **Comparison:** xv6 concept and Linux source landmarks, with revision caveats.
7. **Limits:** untested paths and claims the evidence cannot support.

Do not paste an unexplained command transcript. Every command needs a purpose, prediction, and interpretation.

3. Reconstruct the complete ToyOS path

```
kernel_main chooses EXPECT_FAULT == 2
  → volatile store targets 0x70000000
  → compiler emits a memory-write instruction
  → current page tables have no translation
  → x86 raises #PF, records CR2 and error code
  → IDT vector 14 selects IST1 and isr14
  → stub adds vector; common code saves registers
  → RDI points to uniform interrupt_frame
  → C handler prints vector/error/RIP and decodes CR2
  → expected-fault policy emits PASS and exits QEMU
  → host harness checks status and exact marker
```

Qualify two links carefully. C source alone does not prove the emitted instruction; inspect the fault ELF. Source and static page-table construction support the mapping prediction; the dynamic page-fault output confirms the tested result under QEMU.

4. Compare with xv6 at the concept level

xv6 is valuable because its small trap path exposes the same categories: architecture entry, assembly save area, C dispatcher, cause decode, and kernel policy. But the commonly taught xv6-riscv tree is RISC-V, not x86-64.

Conceptual correspondence:

```
ToyOS IDT gate/stub     ↔ RISC-V stvec trampoline/vector entry
interrupt_frame         ↔ trapframe/register save area
vector + CR2 + error    ↔ scause + stval + sepc
interrupt_handler       ↔ trap dispatcher
panic/expected exit     ↔ kernel policy after cause decode
```

This is not field-for-field equivalence. RISC-V records trap metadata in CSRs and software saves registers. It has no x86 IDT, hardware error-code stack word, or IST.

If a local xv6 source tree is present, record its exact path and revision before citing a function. Otherwise use the current upstream xv6-riscv source and name the revision. Never claim that a remembered filename proves current behavior.

5. Compare with Linux without pretending ToyOS is Linux

Linux's x86 exception path has analogous layers at much greater scale:

- architecture entry assembly under `arch/x86/entry/`;
- x86 trap/fault handling under `arch/x86/kernel/` and `arch/x86/mm/`;

- generic memory-fault policy in `mm/`;
- diagnostic symbolization using kernel symbols and debug information.

Exact files and helper names change. Inspect a current source revision before making a line-level claim. Begin at the architecture entry, identify the `pt_regs`-style saved state, follow page-fault dispatch into memory-management policy, and note where user faults, kernel faults, fixups, signals, and panic paths diverge.

ToyOS has none of Linux's process address spaces, demand paging, exception tables, signals, locking, per-CPU entry stacks, or production recovery policy. Its expected page fault exits the VM.

6. Architecture comparison at the diagnosis boundary

The evidence tuple generalizes:

```
cause + fault address/detail + saved PC + saved registers + exact symbols
```

Its realization differs:

- x86-64: vector/error, `CR2`, saved `RIP`, IDT/IST;
- AArch64: `ESR_ELx`, `FAR_ELx`, `ELR_ELx`, vector-table slot;
- RISC-V: `scause`, `stval`, `sepc`, `stvec`.

The C-level handler structure, naming, and policy are OS implementation. The calling bridge from assembly is an internal kernel ABI layered on the machine architecture.

7. Practical challenge

Use `challenges/day-007-fault-report.sh` from the repository root.

Purpose: generate a reproducible evidence bundle from the canonical Week 6 project without changing it.

Predict: write one expected marker and host status for normal, divide, page, and assertion profiles. Predict the page-fault address and decode.

Run: the wrapper records tool versions, selected source contracts, ordered symbols, disassembly around fault injection, and the complete existing Week 6 test suite into a temporary report file. It prints the report path and key summary lines.

Evidence: retain the generated report if you want a learning artifact; it is intentionally placed under `/tmp`, outside the repository. Add your interpretation separately. Generated output is evidence, not the explanation.

8. Separate observation from inference

Use verbs precisely:

- **source states** a selected IST index;
- **ELF contains** a symbol at an observed address;
- **objdump decodes** bytes as an instruction;
- **QEMU emitted** serial bytes and a host status;

- **GDB observed** a register in a specific stopped VM;
- **we infer** the causal link from these compatible facts.

Avoid “proved the kernel is correct.” A finite set of tests supports only the tested properties under recorded conditions.

9. Current manuals and optional local material

Normative details belong to current architecture and ABI documents:

- Intel 64 and IA-32 SDM or AMD64 Architecture Programmer's Manual;
- System V AMD64 ABI;
- current Multiboot2 and ELF documentation;
- Arm A-profile architecture reference for AArch64 comparison;
- RISC-V privileged specification for xv6/RISC-V comparison.

Use `references/ARCHITECTURE_COMPARISON.md` as a local map. Optional source and lab material under `references/library/` can offer examples, but record provenance and do not let examples override current manuals or source.

Integrated mastery: Explain · Draw · Observe · Build

Explain

Without notes, narrate the page-fault chain from C expression to checked host status. At every boundary say architecture, ABI, or implementation.

Draw

Draw three side-by-side exception paths for ToyOS x86-64, xv6-riscv, and Linux x86-64. Use solid arrows for observed Week 6 evidence and dashed arrows for conceptual comparison. Do not draw identical frame layouts.

Observe

Audit the generated report:

- Does it identify the exact ELF?
- Does it preserve all four expected outcomes?
- Does it include enough source and symbol context?
- Are inferred claims visibly separated?
- Are tool versions and limitations recorded?

Build

Write a two-page incident note answering:

1. What happened?
2. Which instruction and address were involved?
3. How did the machine enter the handler?
4. Which state was hardware-created versus software-created?

5. Why was the result expected?
6. What would a production kernel need beyond ToyOS?
7. Which claim remains least certain, and what next observation would narrow it?

Final bridge

Week 6 has established boot and exception observability. Week 7 can build memory allocation, mappings, interrupts, and kernel-task scheduling on that base. Ring 3, user processes, syscalls, and isolated address spaces are not present yet.

Do not mark progress automatically. Preserve evidence first, then assess mastery yourself.

Now check your understanding

You have finished the theory for this section. Before moving on, use the quick check below to find anything that still feels uncertain, then reinforce the idea with the practical lab when one is provided.

Quick theory check

Answer from memory first. Explanations appear only after you submit your choices.

- Start the interactive theory check
- View quiz source on GitHub

Put it into practice — Fault Report

Format: shell lab

What you will do: Generate a temporary, reproducible Week 6 fault-debug evidence report.

Predict the important result first, then run the lab and explain any difference between prediction and observation.

- Open the lab file

Copy and run:

```
bash 06-toyos-bootstrap-and-exceptions/challenges/day-007-fault-report.sh
```

- View lab source on GitHub